

Ch11: Sorting in Ascending & Descending Order

Slide 1: Understanding Ascending Order

Ascending order means arranging values from the **smallest value to the largest value**.

Think of students standing in a line according to their height, starting with the shortest and ending with the tallest.

Unsorted:

40 10 30 20 50

Ascending:

10 20 30 40 50

When sorting an array, the program usually compares **two neighboring elements** and checks whether they are in the correct order.

For ascending order:

```
if (arr[j] > arr[j + 1])  
{  
    // swap  
}
```

Suppose the array contains:

20 10

The program compares:

20 > 10

This is true.

Since 20 is larger than 10, the smaller value should come first.

So the two values need to exchange positions:

Before:

20 10

After:

10 20

Why >?

For ascending order, we want:

smaller → larger

Therefore, if the **left value is larger than the right value**, they are in the wrong order.

Key idea:

In ascending order, swap when the left element is greater than the right element.

Slide 2: Understanding Descending Order

Descending order means arranging values from the **largest value to the smallest value**.

Unsorted:

40 10 30 20 50

Descending:

50 40 30 20 10

For descending order, the larger values must appear first.

When comparing two neighboring elements, we use:

```
if (arr[j] < arr[j + 1])  
{  
    // swap  
}
```

Suppose:

10 20

The program compares:

10 < 20

This is true.

But for descending order, we want the larger value first:

20 10

Therefore, the values must be swapped.

Why <?

For descending order, we want:

larger → smaller

So if the **left value is smaller than the right value**, they are in the wrong order.

Before:

10 20

After:

20 10

Ascending vs Descending

Order	Desired arrangement	Swap condition
Ascending	Smaller → Larger	<code>arr[j] > arr[j + 1]</code>
Descending	Larger → Smaller	<code>arr[j] < arr[j + 1]</code>

Key idea:

In descending order, swap when the left element is smaller than the right element.

Slide 3: Comparing Two Elements

Sorting becomes much easier to understand when we stop looking at the entire array and focus on **two elements at a time**.

Suppose we have:

20 10

The program compares the two values.

For Ascending Order

We want:

smaller $\rightarrow$ larger

Currently:

20 10

Since 20 is larger than 10, the order is incorrect.

$20 > 10$

So we swap:

10 20

For Descending Order

We want:

larger $\rightarrow$ smaller

The current order is:

20 10

This is already correct.

No swap is required.

Another Example

Consider:

10 20

For ascending:

10 20

Already correct.

For descending:

10 20

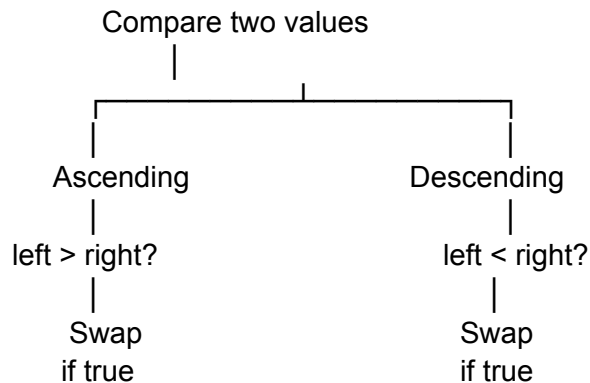
This is incorrect because 10 is smaller than 20.

$10 < 20$

So we swap:

20 10

The Important Pattern



The sorting algorithm does not need to completely change just because the direction changes.

The **comparison condition** changes.

Key idea:

The desired order tells the program when two elements are in the wrong position.

Slide 4: Swapping Two Values

Once the program decides that two elements are in the wrong order, it needs to **exchange their positions**.

Suppose:

`arr[i] = 20`

`arr[j] = 10`

We want:

`arr[i] = 10`

`arr[j] = 20`

A common mistake would be:

```
arr[i] = arr[j];  
arr[j] = arr[i];
```

This does **not** work correctly.

After the first statement:

```
arr[i] = 10  
arr[j] = 10
```

The original 20 has been lost.

We need a temporary storage location.

```
int temp;
```

```
temp = arr[i];  
arr[i] = arr[j];  
arr[j] = temp;
```

Step-by-Step

Before swapping:

```
arr[i] → 20  
arr[j] → 10
```

Step 1: Save `arr[i]`

```
temp = arr[i];
```

Now:

```
temp → 20  
arr[i] → 20  
arr[j] → 10
```

Step 2: Move `arr[j]` into `arr[i]`

```
arr[i] = arr[j];
```

Now:

```
temp → 20  
arr[i] → 10
```

`arr[j] → 10`

Step 3: Put the saved value into `arr[j]`

`arr[j] = temp;`

Now:

`temp → 20`

`arr[i] → 10`

`arr[j] → 20`

Final result:

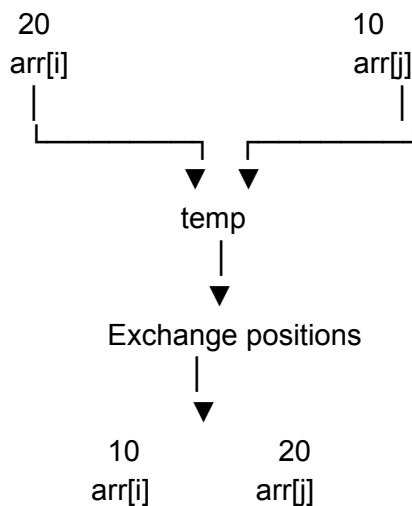
Before:

20 10

After:

10 20

Visual Idea



Key idea:

`temp` protects one value while the two array elements exchange positions.

Slide 5: One Algorithm, Two Directions

Ascending and descending sorting use the same basic process:

Compare → Decide → Swap if needed → Continue

The main difference is the **comparison condition**.

Ascending

Smaller values should come first.

```
if (arr[j] > arr[j + 1])  
{  
    // swap  
}
```

Example:

20 10

Since:

$20 > 10$

Swap:

10 20

Descending

Larger values should come first.

```
if (arr[j] < arr[j + 1])  
{  
    // swap  
}
```

Example:

10 20

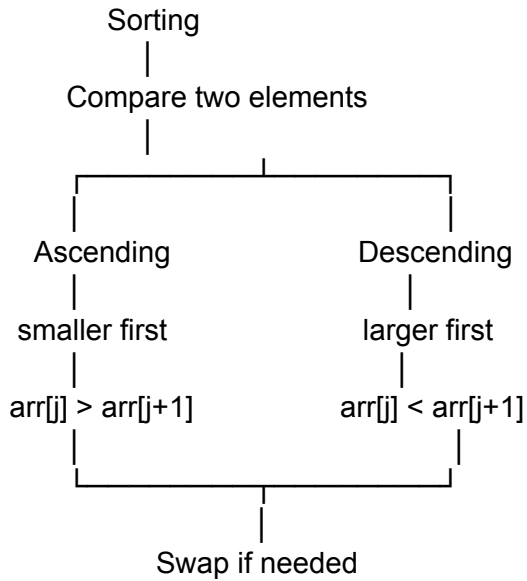
Since:

$10 < 20$

Swap:

20 10

The Big Picture



This is an important idea because the **overall sorting algorithm can remain almost identical**. To change the direction, we often only need to change the comparison.

Ascending:

> → smaller values move toward the beginning

Descending:

< → larger values move toward the beginning

Core Takeaway

Ascending: smaller values come first.

Descending: larger values come first.

Swap: use a temporary variable to safely exchange two values.

Once this comparison-and-swap idea is clear, Bubble Sort becomes much easier to understand, because Bubble Sort repeatedly applies exactly this process to neighboring elements